Giới thiệu về ngôn ngữ Java

Trường ĐH Công nghệ, ĐHQGHN

Nội dung chính

- Lịch sử ngôn ngữ lập trình
- So sánh lập trình thủ tục và lập trình hướng đối tượng
- Lịch sử ngôn ngữ Java
- Môi trường lập trình Java, máy ảo, và JDK
- Giới thiệu về các loại chương trình Java
- Dịch và chạy chương trình

Phần mềm ngày càng phức tạp

- Kích thước ngày càng lớn*
  - LibreOffice: 9M dòng lệnh
  - Android: 13.3M dòng lệnh
  - NetBeans IDE: 97.2M dòng lệnh
- Người dùng ngày càng đòi hỏi nhiều chức năng, đặc biệt là chức năng *thông minh*
- Phần mềm luôn cần được sửa đổi

Nguồn: https://www.openhub.net

Vì vậy

- Cần kiểm soát chi phí
  - Chi phí phát triển
  - Chi phí bảo trì
- Giải pháp chính là ***sử dụng lại***
  - Giảm chi phí và thời gian phát triển
  - Nâng cao chất lượng (được sử dụng lại chứng tỏ có chất lượng tốt)

Để sử dụng lại (mã nguồn)

- Cần dễ hiểu
- Được coi là chính xác
- Có **giao diện** rõ ràng
- Không/ít yêu cầu thay đổi khi sử dụng trong phần mềm mới
- Có thể mở rộng nếu cần thiết

Các phương pháp lập trình

- Lập trình không có cấu trúc
- Lập trình có cấu trúc (lập trình thủ tục)
- Lập trình logic
- Lập trình hàm
- Lập trình hướng đối tượng

Lập trình không có cấu trúc
(non-structured programming)

- Là phương pháp xuất hiện đầu tiên
  - các ngôn ngữ như Assembly, Basic
  - sử dụng các biến tổng thể
  - lạm dụng lệnh GOTO
- Các nhược điểm
  - khó hiểu, khó bảo trì, hầu như không thể sử dụng lại
  - chất lượng kém
  - chi phí cao
  - không thể phát triển các ứng dụng lớn

Ví dụ

```
10    k =1
20    gosub 100
30    if y > 120 goto 60
40    k = k+1
50    goto 20
60    print k, y
70    stop
100   y = 3*k*k + 7*k-3
110   return
```

Lập trình logic

```
grandparent(X,Z) :- parent(X,Y), parent(Y,Z).
parent(X,Y):- father(X,Y).
parent(X,Y):- mother(X,Y).

father(john,lily).
mother(kathy,lily).
mother(lily,bill).
father(ken,karen).
```

?-parent(lily,bill)

?-grandparent(Q,bill)

Lập trình có cấu trúc/lập trình thủ tục
(structured/procedural programming)

- Sử dụng các lệnh có cấu trúc: for, do while, if then else...
- Các ngôn ngữ: Pascal, C, ...
- Chương trình là tập các hàm/thủ tục
- Ưu điểm
  - Chương trình được cục bộ hóa, do đó dễ hiểu, dễ bảo trì hơn
  - Dễ dàng tạo ra các thư viện phần mềm

Ví dụ

```
struct Date {
    int year, mon, day;
};
...
print_date(Date d) {
    printf(”%d / %d / %d\n”, d.day,
           d.mon, d.year);
}
```

Lập trình có cấu trúc/lập trình thủ tục

- Nhược điểm
  - dữ liệu và mã xử lý là tách rời
  - người lập trình phải biết cấu trúc dữ liệu (vấn đề này một thời gian dài được coi là hiển nhiên)
  - khi thay đổi cấu trúc dữ liệu thì mã xử lý (thuật toán) phải thay đổi theo
  - khó đảm bảo tính đúng đắn của dữ liệu
  - không tự động khởi tạo hay giải phóng dữ liệu động

Tại sao phải thay đổi cấu trúc dữ liệu?

- Cấu trúc dữ liệu là mô hình của bài toán cần giải quyết
  - Do thiếu kiến thức về bài toán, về miền ứng dụng..., không phải lúc nào cũng tạo được cấu trúc dữ liệu hoàn thiện ngay từ đầu.
  - Tạo ra một cấu trúc dữ liệu hợp lý luôn là vấn đề đau đầu của người lập trình.
- Bản thân bài toán cũng không bất biến
  - Cần phải thay đổi cấu trúc dữ liệu để phù hợp với các yêu cầu thay đổi.

Các vấn đề

- Thay đổi cấu trúc
  - dẫn đến việc sửa lại mã chương trình (thuật toán) tương ứng và làm chi phí phát triển tăng cao.
  - không tái sử dụng được các mã xử lý ứng với cấu trúc dữ liệu cũ.
- Đảm bảo tính đúng đắn của dữ liệu
  - một trong những nguyên nhân chính gây ra lỗi phần mềm là gán các dữ liệu không hợp lệ
  - cần phải kiểm tra tính đúng đắn của dữ liệu mỗi khi thay đổi giá trị

Ví dụ: MyDate

MyDate.java:

```
class MyDate {
    public int year, month, day;
}
```

MyCalendar.java:

```
MyDate d = new MyDate();
d.day = 32;                    // invalid day

d.day = 31; d.month = 2; // how to check
d.day = d.day + 1;             //
```

Ví dụ: MyDate (2)

Thay đổi cấu trúc dữ liệu:

MyDate.java:

```
class MyDate {
    public short year;
    public short mon_n_day;
}
```

Giải pháp

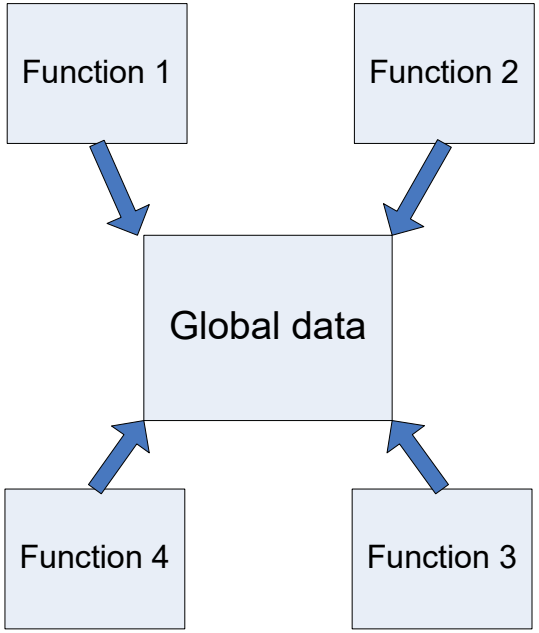
- Che giấu dữ liệu (che giấu cấu trúc)
- Truy cập dữ liệu thông qua giao diện xác định

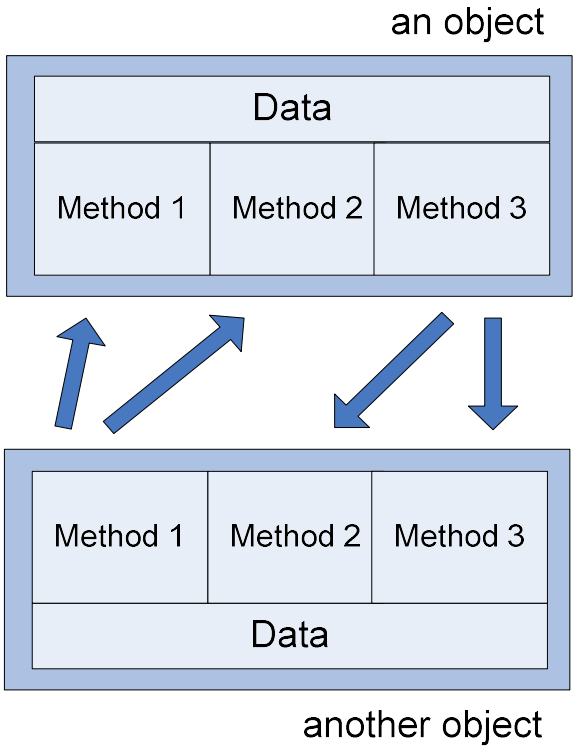
```
class MyDate {
    private int year, mon, day;
    public int getDay() {...}
    public boolean setDay(int)
  {...}
    ...
}
```

Sử dụng giao diện

MyCalendar.java:

```
MyDate d = new MyDate();
...
d.day = 32;     // error
d.setDay(31);
d.setMonth(2); // should return False
```


Function 1
Function 2
Global data
Function 4
Function 3


an object
Data
Method 1
Method 2
Method 3
Method 1
Method 2
Method 3
Data
another object

Đóng gói và che giấu thông tin

- Đóng gói: Đóng gói dữ liệu và các thao tác tác động lên dữ liệu thành một thể thống nhất (lớp đối tượng) thuận tiện cho sử dụng lại
- Che giấu thông tin
  - thao tác với dữ liệu thông qua các giao diện xác định
  - che giấu người *lập trình khách* (*client programmer*) cái có khả năng thay đổi (tách cái bất biến ra khỏi cái khả biến)

Lớp và đối tượng

- Lớp đối tượng (class) là khuôn mẫu để sinh ra đối tượng
- Đối tượng là thể hiện (instance) của một lớp. Đối tượng có
  - thuộc tính (dữ liệu)
  - hành vi (phương thức)

Hệ thống hướng đối tượng

- Bao gồm một tập các đối tượng
  - mỗi đối tượng chịu trách nhiệm một công việc
- Các đối tượng tương tác thông qua trao đổi thông điệp (*message passing*)
- Các đối tượng có thể tồn tại phân tán/có thể hoạt động song song

Mô hình hóa đối tượng

MyDate
-year -month -day
+ getDay() + setDay(int) + getMonth() + setMonth(int) + getYear() + setYear(int) - validDate(int, int, int)

Lợi ích của lập trình hướng đối tượng

- năng suất lập trình (năng suất phát triển)
- chất lượng phần mềm
- tính hiểu được của phần mềm
- vòng đời của phần mềm

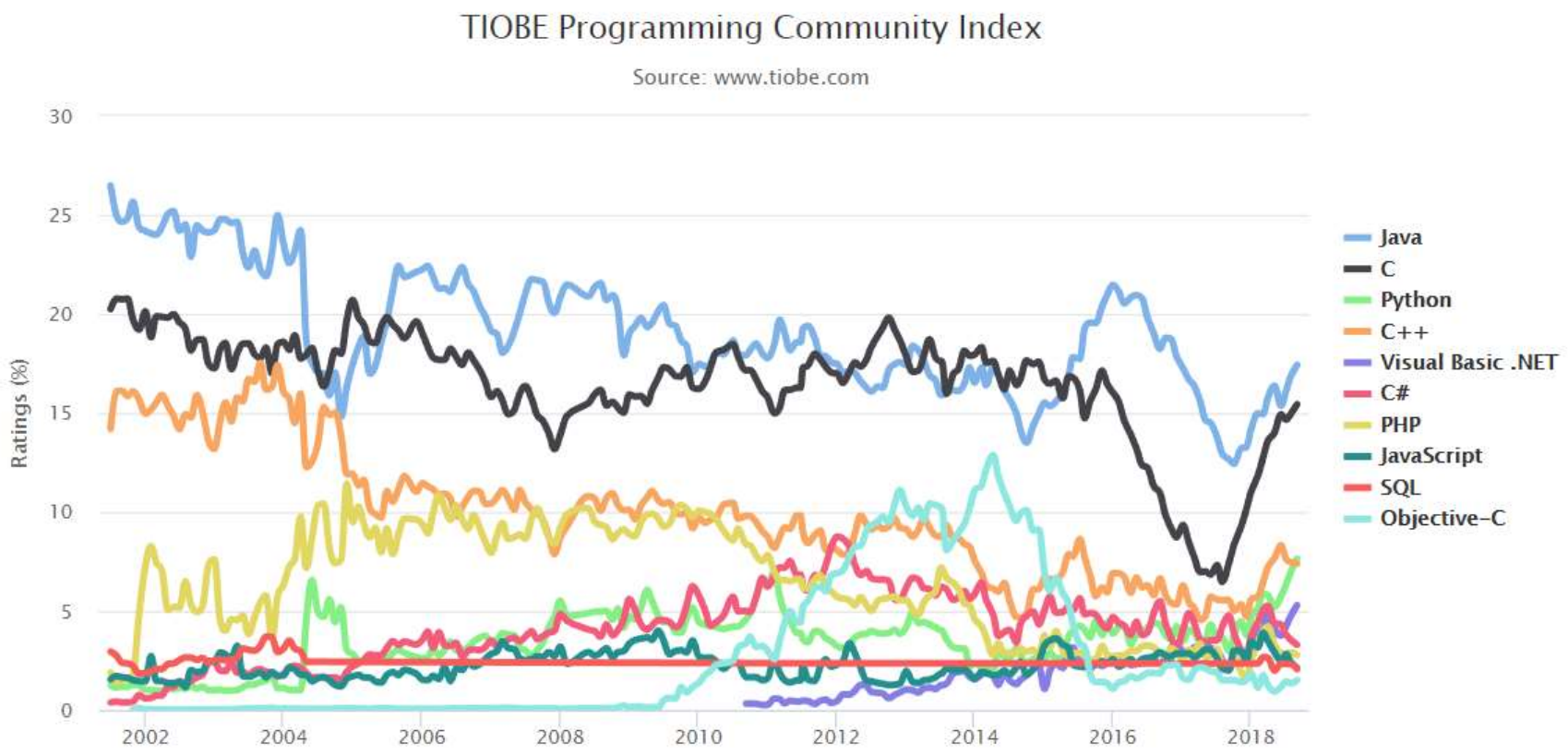
OOP và OOL

- Có thể thể hiện phần nào tư tưởng đóng gói/che giấu thông tin trên ngôn ngữ thủ tục
  - không triệt để, khó kiểm soát
- Ngôn ngữ hướng đối tượng cung cấp khả năng kiểm soát truy cập; ngoài ra
  - kế thừa
  - đa hình

Java

- 1991: phát triển bởi Sun Microsystems như là một ngôn ngữ lập trình cho môi trường nhúng
  - Tên ban đầu là Oak
- Java 1.0.2, 1.1
  - "Write Once, Run Anywhere"
  - Chậm
  - Được sử dụng cho các ứng dụng Web (applets)
- Java 2 (versions 1.2 - 1.4)
  - Nhanh và mạnh hơn,
  - 3 nền tảng: J2ME, J2SE, J2EE
- Java 5, 6, 7, 8 (versions 1.5-1.6…)
  - Được cải tiến nhiều hơn nữa

Thống kê về ngôn ngữ lập trình



Thống kê của IEEE

Web | Mobile | Enterprise | Embedded

Language Rank	Types	Spectrum Ranking
1. Python	Web, Enterprise, Embedded	100.0
2. C++	Mobile, Enterprise, Embedded	99.7
3. Java	Web, Mobile, Enterprise	97.5
4. C	Mobile, Enterprise, Embedded	96.7
5. C#	Web, Mobile, Enterprise	89.4
6. PHP	Web	84.9
7. R	Enterprise	82.9
8. JavaScript	Web, Mobile	82.6
9. Go	Web, Enterprise	76.4
10. Assembly	Embedded	74.1

Thống kê của IEEE (theo nhu cầu tuyển dụng)

Web | Mobile | Enterprise | Embedded

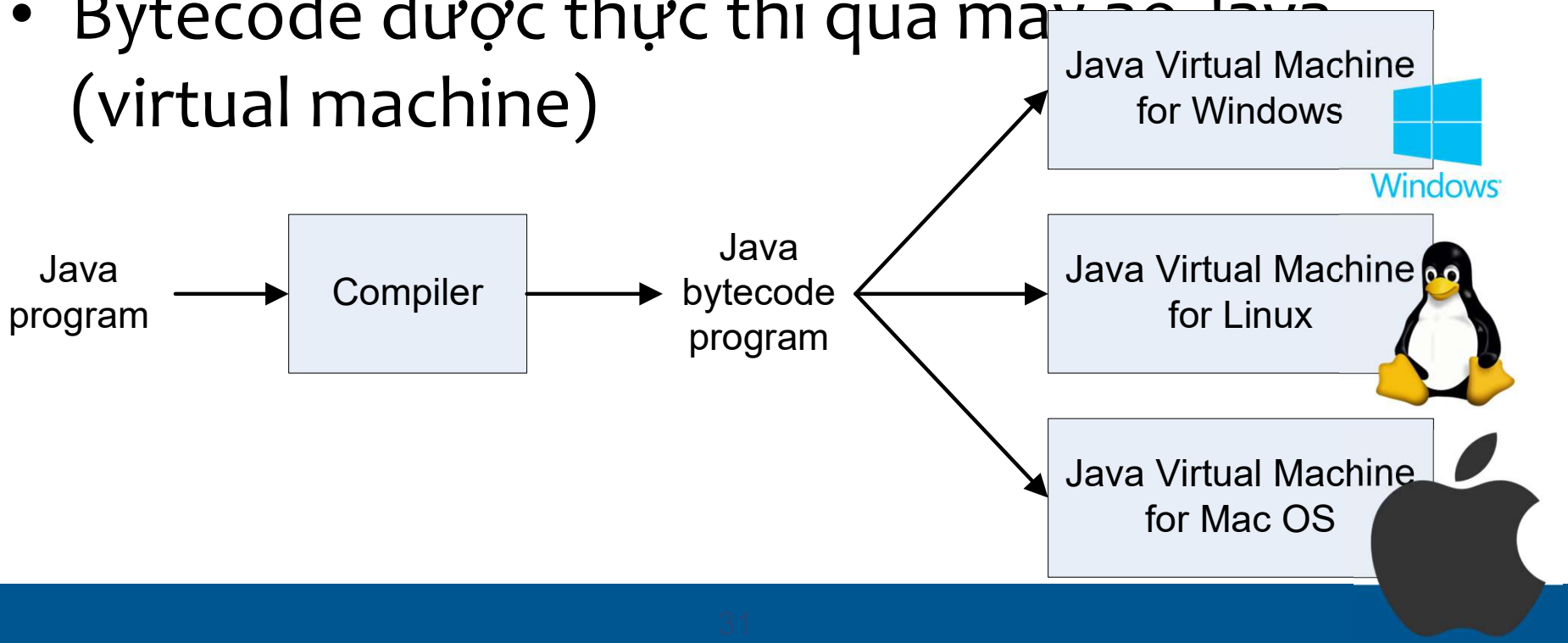
Language Rank	Types	Jobs Ranking
1. Python	Web, Enterprise, Embedded	100.0
2. Java	Web, Mobile, Enterprise	99.2
3. C	Mobile, Enterprise, Embedded	98.8
4. C++	Mobile, Enterprise, Embedded	94.6
5. C#	Web, Mobile, Enterprise	86.2
6. JavaScript	Web, Mobile	85.7
7. Assembly	Embedded	83.4
8. PHP	Web	83.1
9. HTML	Web	81.3
10. Scala	Web, Mobile	76.5

Cấu trúc của chương trình Java

- Tập hợp của các lớp đối tượng
- Thông thường mỗi lớp được mô tả trong 1 tệp mã nguồn (trùng tên với tên lớp)
  - Tăng tính độc lập
  - Dễ sửa đổi, tiết kiệm thời gian biên dịch

Biên dịch

- Mã nguồn Java được biên dịch thành mã bytecode
- Bytecode độc lập với nền tảng
- Bytecode được thực thi qua máy ảo Java (virtual machine)



Java Virtual Machine (JVM)

- JVM được hiện thực phụ thuộc vào các nền tảng (hardware, OS)
- Đảm bảo các chương trình Java (bytecode) có thể thực thi trên các nền tảng khác nhau (platform-independent)
- Đảm bảo an ninh
- Thường được hiện thực như là một phần mềm
  - JRE - Java Runtime Environment
- Java platform: JVM + APIs

Các loại ứng dụng Java

- Ứng dụng để bàn – *Java Standard Edition (Java SE)*
- Ứng dụng phân tán, ứng dụng chủ – *Java Enterprise Edition (Java EE)*
- Ứng dụng di động – *Java Micro Edition (Java ME)*
- Ứng dụng trên thẻ - Java Card

HelloWorld application

trùng tên lớp

HelloWorld.java:

Lớp *Tên lớp*

```
public class HelloWorld {

  public static void main (String[] args) {

    System.out.println("Hello, world");

  }
}
```

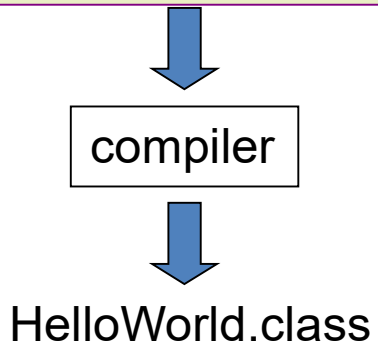
Tên phương thức

Phát biểu trong phương thức

public, từ khóa quy định mức truy xuất

Biên dịch và thực thi

```
public class HelloWorld {
  public static void main (String[] args)
  {
    System.out.println("Hello, world");
  }
}
```



- Compile HelloWorld.java

```
javac HelloWorld.java
```

- Run

```
java HelloWorld
```

```
%> javac HelloWorld.java
%> java HelloWorld
Hello, world
```

Ứng dụng với nhiều hơn một lớp

2 lớp được hiện thực bởi 2 file

TestGreeting.java:

```
public class TestGreeting {
  public static void main(String[] args) {
    Greeting gr = new Greeting();
    gr.greet();
  }
}
```

Greeting.java:

```
public class Greeting {
  public void greet() {
    System.out.print("Hi there!");
  }
}
```

Biên dịch và thực thi

- Compile

`javac TestGreeting.java`

  - `Greeting.java` được dịch tự động

- Run

`java TestGreeting`

```
%> javac TestGreeting.java
%> java TestGreeting
Hi there!
```

JDK – Java Development Kit

- Môi trường phát triển ứng dụng Java
- Các thành phần chính
  - *javac:* trình biên dịch, chuyển mã nguồn Java thành bytecode
  - *java:* trình thông dịch
  - *javadoc:* công cụ sinh tài liệu từ các chú thích trong mã nguồn
  - *jdb:* trình gỡ lỗi